Vulkan Devices

Matthew Lefler — @skye

October 22, 2026

1 The problem

I want to be able to input a chain of structures that define what I want a device to support. Then the program returns a logical device that has that support.

2 The issue

The list of structures is very long, fix by parsing *vk.xml*.

Though the main problem is that compairing two structures is nigh impossible based on *vk.xml*

2.1 Prob. 1

It is not possible to tell which items have been set or not.

Example:

```
VkPhysicalDeviceFeatures2 device_features_2 = {
    .features.geometryShader = VK_TRUE,
    .sType = VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_FEATURES_2,
};
```

Results in:

```
VkPhysicalDeviceFeatures2 device_features_2 = {
    VkStructureType sType = VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_FEATURES_2;
    void* pNext = 0;
    VkPhysicalDeviceFeatures features = {
        VkBool32   robustBufferAccess = VkFalse;
        VkBool32   fullDrawIndexUint32 = VkFalse;
        VkBool32   imageCubeArray = VkFalse;
        VkBool32   independentBlend = VkFalse;
        VkBool32   geometryShader = VkTrue;
        VkBool32   tessellationShader = VkFalse;
        VkBool32   sampleRateShading = VkFalse;
        VkBool32   dualSrcBlend = VkFalse;
        VkBool32   logicOp = VkFalse;
        VkBool32   multiDrawIndirect = VkFalse;
        VkBool32   drawIndirectFirstInstance = VkFalse;
        VkBool32   depthClamp = VkFalse;
        VkBool32   depthBiasClamp = VkFalse;
```

```
18          VkBool32   fillModeNonSolid = VkFalse;
19          VkBool32   depthBounds = VkFalse;
20          VkBool32   wideLines = VkFalse;
21          VkBool32   largePoints = VkFalse;
22          VkBool32   alphaToOne = VkFalse;
23          VkBool32   multiViewport = VkFalse;
24          VkBool32   samplerAnisotropy = VkFalse;
25          VkBool32   textureCompressionETC2 = VkFalse;
26          VkBool32   textureCompressionASTC_LDR = VkFalse;
27          VkBool32   textureCompressionBC = VkFalse;
28          VkBool32   occlusionQueryPrecise = VkFalse;
29          VkBool32   pipelineStatisticsQuery = VkFalse;
30          VkBool32   vertexPipelineStoresAndAtomics = VkFalse;
31          VkBool32   fragmentStoresAndAtomics = VkFalse;
32          VkBool32   shaderTessellationAndGeometryPointSize = VkFalse;
33          VkBool32   shaderImageGatherExtended = VkFalse;
34          VkBool32   shaderStorageImageExtendedFormats = VkFalse;
35          VkBool32   shaderStorageImageMultisample = VkFalse;
36          VkBool32   shaderStorageImageReadWithoutFormat = VkFalse;
37          VkBool32   shaderStorageImageWriteWithoutFormat = VkFalse;
38          VkBool32   shaderUniformBufferArrayDynamicIndexing = VkFalse;
39          VkBool32   shaderSampledImageArrayDynamicIndexing = VkFalse;
40          VkBool32   shaderStorageBufferArrayDynamicIndexing = VkFalse;
41          VkBool32   shaderStorageImageArrayDynamicIndexing = VkFalse;
42          VkBool32   shaderClipDistance = VkFalse;
43          VkBool32   shaderCullDistance = VkFalse;
44          VkBool32   shaderFloat64 = VkFalse;
45          VkBool32   shaderInt64 = VkFalse;
46          VkBool32   shaderInt16 = VkFalse;
47          VkBool32   shaderResourceResidency = VkFalse;
48          VkBool32   shaderResourceMinLod = VkFalse;
49          VkBool32   sparseBinding = VkFalse;
50          VkBool32   sparseResidencyBuffer = VkFalse;
51          VkBool32   sparseResidencyImage2D = VkFalse;
52          VkBool32   sparseResidencyImage3D = VkFalse;
53          VkBool32   sparseResidency2Samples = VkFalse;
54          VkBool32   sparseResidency4Samples = VkFalse;
55          VkBool32   sparseResidency8Samples = VkFalse;
56          VkBool32   sparseResidency16Samples = VkFalse;
57          VkBool32   sparseResidencyAliased = VkFalse;
58          VkBool32   variableMultisampleRate = VkFalse;
59          VkBool32   inheritedQueries = VkFalse;
60      };
61  };
```

So how does one tell which fields were set on purpose, because while in this case only chencking for true values makes sense, it is completely possible that a structure has a field that requires the corresponding value from the device's

feature's to also be false and not true.

3 Prob. 2

And even more pressing a value may be left as uninitilized, resulting in any possible value technically. This makes the field impossible to be used, and therefore requires knowing which fields were set.

4 The fix

Notice that every structure that extends "VkPhysicalDeviceFeatures2" only contains the "sType", "pNext", and some number of "VkBool32" fields

4 months ago...